

Assignment 1

Daniel Mehta

```
In [11]: import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [12]: df = pd.read_csv("bc.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 699 entries, 0 to 698
Data columns (total 11 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Sample code number                    699 non-null    int64
 1   Clump Thickness                       699 non-null    int64
 2   Uniformity of Cell Size               699 non-null    int64
 3   Uniformity of Cell Shape              699 non-null    int64
 4   Marginal Adhesion                    699 non-null    int64
 5   Single Epithelial Cell Size           699 non-null    int64
 6   Bare Nuclei                          699 non-null    object
 7   Bland Chromatin                      699 non-null    int64
 8   Normal Nucleoli                      699 non-null    int64
 9   Mitoses                             699 non-null    int64
10   Class Label                          699 non-null    int64
dtypes: int64(10), object(1)
memory usage: 60.2+ KB
```

```
In [13]: df['Bare Nuclei'] = pd.to_numeric(df['Bare Nuclei'], errors='coerce')
df.dropna(inplace=True)
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 683 entries, 0 to 698
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Sample code number                    683 non-null    int64
1   Clump Thickness                       683 non-null    int64
2   Uniformity of Cell Size               683 non-null    int64
3   Uniformity of Cell Shape              683 non-null    int64
4   Marginal Adhesion                    683 non-null    int64
5   Single Epithelial Cell Size           683 non-null    int64
6   Bare Nuclei                          683 non-null    float64
7   Bland Chromatin                      683 non-null    int64
8   Normal Nucleoli                      683 non-null    int64
9   Mitoses                             683 non-null    int64
10  Class Label                          683 non-null    int64
dtypes: float64(1), int64(10)
memory usage: 64.0 KB
```

Out[13]:

	Sample code number	Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	Bland Chromatin	Normal Nucleoli	Mitoses
count	6.830000e+02	683.000000	683.000000	683.000000	683.000000	683.000000	683.000000	683.000000	683.000000	683.0000
mean	1.076720e+06	4.442167	3.150805	3.215227	2.830161	3.234261	3.544656	3.445095	2.869693	1.6032
std	6.206440e+05	2.820761	3.065145	2.988581	2.864562	2.223085	3.643857	2.449697	3.052666	1.7326
min	6.337500e+04	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.0000
25%	8.776170e+05	2.000000	1.000000	1.000000	1.000000	2.000000	1.000000	2.000000	1.000000	1.0000
50%	1.171795e+06	4.000000	1.000000	1.000000	1.000000	2.000000	1.000000	3.000000	1.000000	1.0000
75%	1.238705e+06	6.000000	5.000000	5.000000	4.000000	4.000000	6.000000	5.000000	4.000000	1.0000
max	1.345435e+07	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.000000	10.0000

Question 1

```
In [14]: X = df.drop('Class Label', axis=1)
y = df['Class Label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

Train size: (546, 10)
Test size: (137, 10)

Question 2

```
In [15]: model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("-"*55)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Confusion Matrix:

```
[[77  2]
 [ 6 52]]
```

Classification Report:

	precision	recall	f1-score	support
2	0.93	0.97	0.95	79
4	0.96	0.90	0.93	58
accuracy			0.94	137
macro avg	0.95	0.94	0.94	137
weighted avg	0.94	0.94	0.94	137

Question 3

```
In [16]: X_cluster = df.drop('Class Label', axis=1)

kmeans = KMeans(n_clusters=2, n_init=10, random_state=42)
df['Cluster Label'] = kmeans.fit_predict(X_cluster)

true_labels = df['Class Label'].map({2: 0, 4: 1})
pred_labels = df['Cluster Label']

print("Confusion Matrix:\n", confusion_matrix(true_labels, pred_labels))
print("-"*55)
print("\nClassification Report:\n", classification_report(true_labels, pred_labels, zero_division=1))
```

Confusion Matrix:

```
[[442  2]
 [239  0]]
```

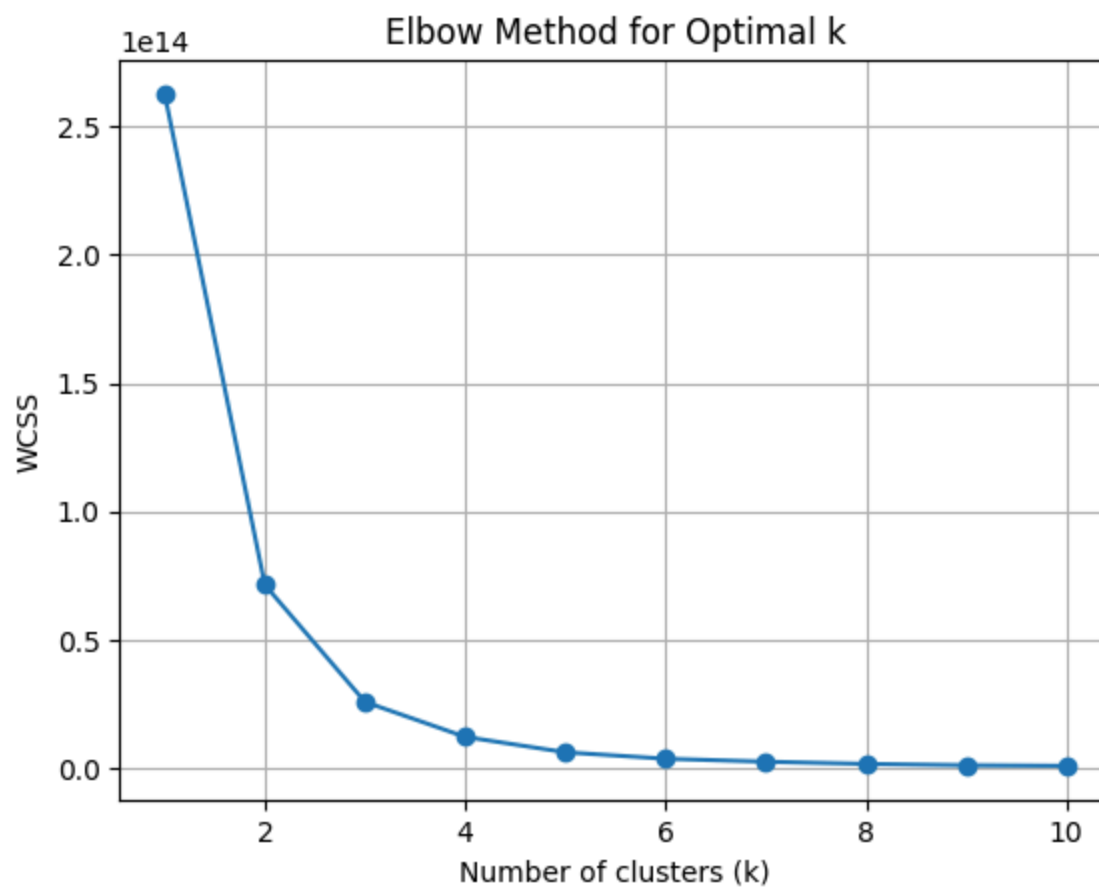
Classification Report:

	precision	recall	f1-score	support
0	0.65	1.00	0.79	444
1	0.00	0.00	1.00	239
accuracy			0.65	683
macro avg	0.32	0.50	0.89	683
weighted avg	0.42	0.65	0.86	683

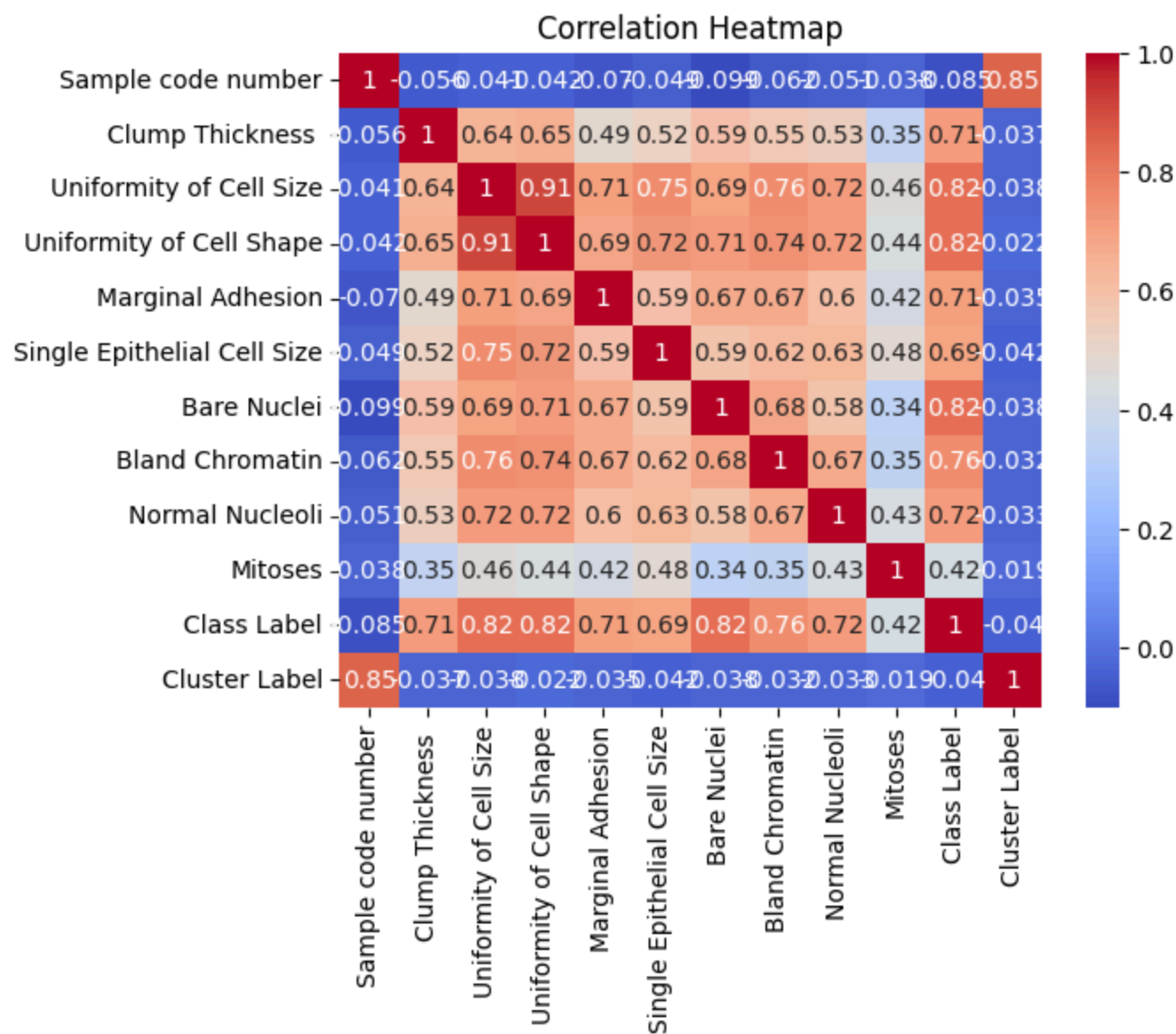
Question 4

```
In [17]: wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    km.fit(X_cluster)
    wcss.append(km.inertia_)

plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```



```
In [18]: sns.heatmap(df.corr(), annot=True, cmap='coolwarm')  
plt.title("Correlation Heatmap")  
plt.show()
```



```
In [19]: print("Final Dataset Shape:", df.shape)
print("-"*55)
print("Class Distribution:\n", df['Class Label'].value_counts())
```

Final Dataset Shape: (683, 12)

Class Distribution:

Class Label

2 444

4 239

Name: count, dtype: int64

Final Summary

- I cleaned the dataset by converting the `Bare Nuclei` column to numeric and removing rows with missing values.
- I trained a logistic regression model which gave around 94% accuracy, meaning it predicted the test data quite well.
- I used KMeans clustering with 2 clusters, but the results didn't match the true labels very well.
- The elbow method helped visualize the best number of clusters, and the heatmap showed how some features are related.

Conclusion

Logistic regression worked well for this dataset, but clustering wasn't as accurate. Overall, supervised learning gave better results.

In []: